Azure DataBricks and Azure Data Factory ETL Project

Name: Vemula Raghunath Goud

Project: End-to-End Data Engineering Pipeline using Azure Data Factory & Databricks

Contents

1. Project Overview 2

2. Data Ingestion 3

3. Metadata-Driven Framework 3

4. Silver Layer (Data Transformation) 5

5. Gold Layer (Aggregations) 6

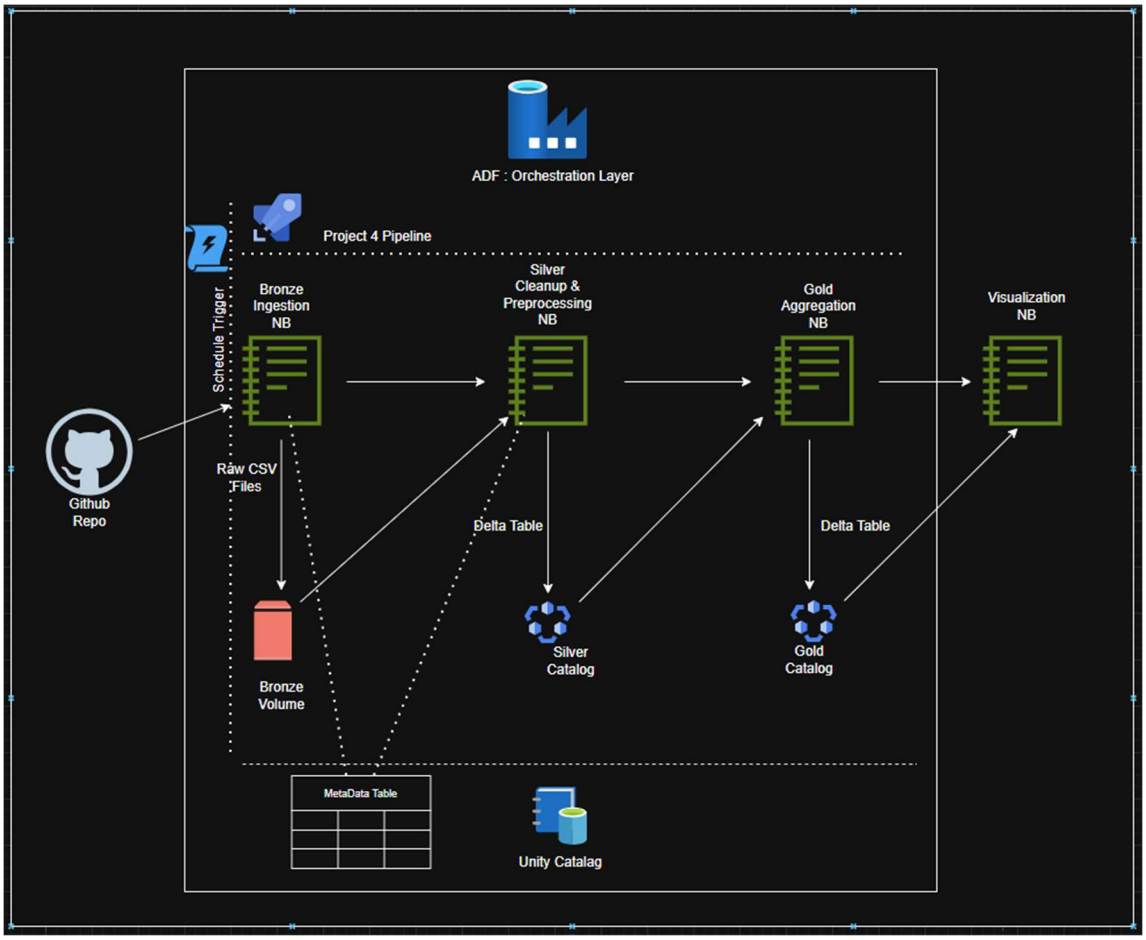
6. Visualization (Plotly) 9

7. ADF Pipeline (Orchestration) 12

8. Data Model 13

9. Challenges & Solutions 14

13. Conclusion 14



Architecture Overview

High-Level Flow

GitHub CSV Files
↓
ADF Pipeline (Orchestration + Schedule)
↓
Databricks Notebook (Ingestion)
↓
Unity Catalog Volume (Bronze - Raw Data)
↓
Metadata Delta Table
↓
Dynamic Notebook (Bronze → Silver)
↓
Silver Delta Table (Cleaned Data)
↓
Databricks Notebook (Gold Aggregations)
↓
Gold Delta Tables (Business Metrics)
↓
Plotly Visualization

1. Project Overview

Objective:
To design and implement a scalable data pipeline that ingests data from GitHub, processes it using Databricks, and produces analytics-ready datasets using a metadata-driven approach.

Technologies Used

Layer	Technology
Orchestration	Azure Data Factory
Storage	ADLS Gen2 & Manged Delta Tables (via Unity Catalog Volume)
Compute	Azure Databricks
Processing	PySpark
Data Format	Delta Lake
Visualization	Plotly
Source	GitHub CSV

2. Data Ingestion

- Source data was fetched from GitHub:
  - https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2019.csv
  - https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2020.csv
  - https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2021.csv
- Data was downloaded using Python (requests) and stored in bronze volume:

```
dbutils.fs.mkdirs("/Volumes/adbrag/project4_schema/project4_volume/raw/")
```

Made directory in volume as :

/Volumes/adbrag/project4_schema/project4_volume/raw/

```
for file_name, url in files.items():
    response = requests.get(url)
    response.raise_for_status() #no file warning

    with open(base_path + file_name, "wb") as f: #writing binary
        f.write(response.content)

    print(f"Downloaded {file_name}")
```

- No transformations were applied in Bronze layer.

3. Metadata-Driven Framework

A metadata table was created to enable dynamic data processing.

Metadata Table

```
CREATE TABLE IF NOT EXISTS adbrag.project4_schema.file_metadata (
  source_name STRING,
  source_file_path STRING,
  target_table STRING,
  file_format STRING,
  delimiter STRING,
  has_header BOOLEAN,
  column_names STRING,
  load_type STRING,
  source_year STRING,
  is_active BOOLEAN,
```

```
    created_ts TIMESTAMP
)
USING DELTA;
```

Inserted values as:

```
INSERT INTO file_metadata
VALUES
(
    'sales_2019',
    '/Volumes/adbrag/project4_schema/project4_volume/raw/2019.csv',
    'sales_silver',
    'csv',
    ',',
    false,
    'SalesOrderNumber,SalesOrderLineNumber,OrderDate,CustomerName,EmailAddress,Prod
uctName,OrderQuantity,UnitPrice,TaxAmount',
    'full',
    '2019',
    true,
    current_timestamp()
),…
```

Metadata is read as:

```
metadata_df = (
    spark.table("adbrag.project4_schema.file_metadata")
        .filter("is_active = true")
)
```

Key Features

- Stores file paths and configurations
- Stores column names for schema assignment
- Enables dynamic ingestion without hardcoding
- Supports scalability for future datasets

4. Silver Layer (Data Transformation)

- Data read from Bronze volume
- Column names assigned dynamically from metadata
- Data types standardized:
    - dates → DateType
    - numeric columns → int/double

Example Transformations

- Renamed _c0, _c1 → business column names
- Added audit columns:
    - SourceYear
    - SourceName
    - LoadTimestamp

Silver transformation code:

```
for row in metadata_df.collect():
#Metadata read in for setting parameters
  source_name = row["source_name"]
  source_file_path = row["source_file_path"]
  target_table = row["target_table"]
  file_format = row["file_format"]
  delimiter = row["delimiter"]
  has_header = row["has_header"]
  source_year = row["source_year"]
  column_names = [c.strip() for c in row["column_names"].split(",")]
#Data read into dataframe
  df = (
    spark.read.format(file_format)
    .option("header", str(has_header).lower())
    .option("delimiter", delimiter)
    .option("quote", '"')
    .load(source_file_path)
  )
#Data selected for all column names as in metadata table
  df = df.toDF(*column_names)
```

```
#DataTypes handled and audit columns added
  df = (
    df.withColumn("SalesOrderLineNumber", col("SalesOrderLineNumber").cast("int"))
      .withColumn("OrderDate", to_date(col("OrderDate"), "yyyy-MM-dd"))
      .withColumn("OrderQuantity", col("OrderQuantity").cast("int"))
      .withColumn("UnitPrice", col("UnitPrice").cast("double")) #ROUND or DECIMAL(10,2)
      .withColumn("TaxAmount", col("TaxAmount").cast("double"))
      .withColumn("SourceYear", lit(source_year))
      .withColumn("SourceName", lit(source_name))
      .withColumn("LoadTimestamp", current_timestamp())
  )
#Data written to silver cleaned tables
  df.write.format("delta") \
    .mode("append") \
    .saveAsTable(f"adbrag.project4_schema.{target_table}")

  print(f"Loaded {source_name} into adbrag.project4_schema.{target_table}")
```

#Output Table

adbrag.project4_schema.sales_silver

5. Gold Layer (Aggregations)

Multiple Gold tables were created for analytics.

1. Product-Year Sales

gold_product_year_sales

Metrics:

- TotalOrders
- TotalQuantity
- TotalSalesAmount
- TotalTaxAmount

```
#Product-Year Sales
gold_product_year = (
  silver_df.groupBy("SourceYear", "ProductName")
  .agg(
    countDistinct("SalesOrderNumber").alias("TotalOrders"),
    sum("OrderQuantity").alias("TotalQuantity"),
```

```
        round(sum(col("OrderQuantity") * col("UnitPrice")), 2).alias("TotalSalesAmount"),
        round(sum("TaxAmount"), 2).alias("TotalTaxAmount")
    )
)
gold_product_year.write.format("delta").mode("overwrite").saveAsTable("adbrag.project4_schema.gold_product_year_sales")
```

2. Customer Sales

gold_customer_sales

Metrics:

- TotalOrders
- TotalQuantity
- TotalSalesAmount

```
# Customer Sales
gold_customer_sales = (
    silver_df.groupBy("CustomerName", "EmailAddress")
    .agg(
        countDistinct("SalesOrderNumber").alias("TotalOrders"),
        sum("OrderQuantity").alias("TotalQuantity"),
        round(sum(col("OrderQuantity") * col("UnitPrice")), 2).alias("TotalSalesAmount"),
        round(sum("TaxAmount"), 2).alias("TotalTaxAmount")
    )
)
gold_customer_sales.write.format("delta").mode("overwrite").saveAsTable("adbrag.project4_schema.gold_customer_sales")
```

3. Yearly Summary

gold_yearly_summary

Metrics:

- TotalOrders
- TotalQuantity
- TotalSalesAmount
- AverageUnitPrice

 # Yearly Summary

```
gold_yearly_summary = (
    silver_df.groupBy("SourceYear")
    .agg(
        countDistinct("SalesOrderNumber").alias("TotalOrders"),
        sum("OrderQuantity").alias("TotalQuantity"),
        round(sum(col("OrderQuantity") * col("UnitPrice")), 2).alias("TotalSalesAmount"),
        round(sum("TaxAmount"), 2).alias("TotalTaxAmount"),
        round(avg("UnitPrice"), 2).alias("AverageUnitPrice")
    )
)
gold_yearly_summary.write.format("delta").mode("overwrite").saveAsTable("adbrag.project
4_schema.gold_yearly_summary")
```

4. Monthly Sales

gold_monthly_sales

Metrics:

- Monthly sales trends
- Total revenue per month

 # Monthly Sales

```
gold_monthly_sales = (
    silver_df.withColumn("OrderMonth", date_format(col("OrderDate"), "yyyy-MM"))
    .groupBy("OrderMonth")
    .agg(
        countDistinct("SalesOrderNumber").alias("TotalOrders"),
        sum("OrderQuantity").alias("TotalQuantity"),
        round(sum(col("OrderQuantity") * col("UnitPrice")), 2).alias("TotalSalesAmount"),
        round(sum("TaxAmount"), 2).alias("TotalTaxAmount")
    )
)
gold_monthly_sales.write.format("delta").mode("overwrite").saveAsTable("adbrag.project4
_schema.gold_monthly_sales")
```

6. Visualization (Plotly)

Plotly was used to visualize Gold data.

```
import plotly.express as px
```

Charts Created

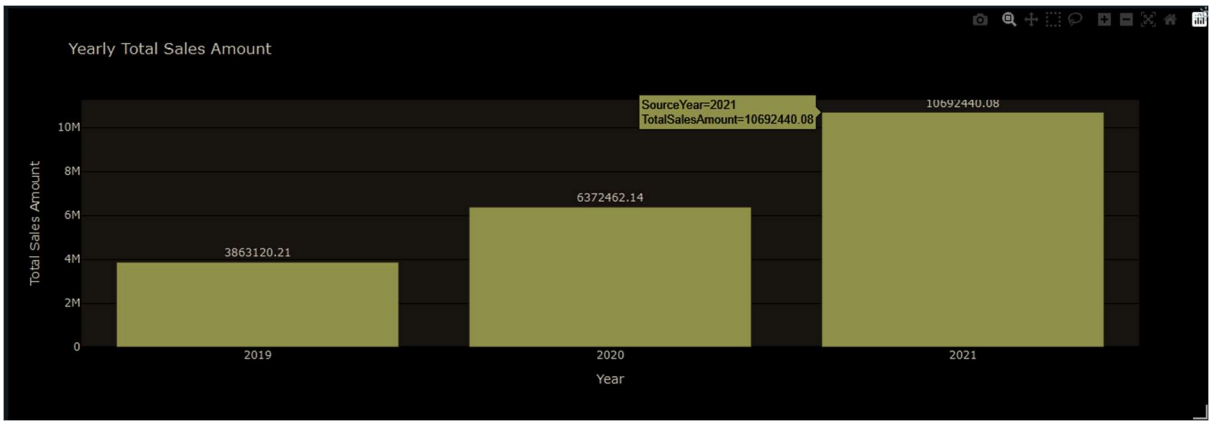
1. **Yearly Sales Bar Chart**

```
df_yearly = spark.table("adbrag.project4_schema.gold_yearly_summary")
pdf_yearly = df_yearly.orderBy("SourceYear").toPandas()

fig = px.bar(
    pdf_yearly,
    x="SourceYear",
    y="TotalSalesAmount",
    text="TotalSalesAmount",
    title="Yearly Total Sales Amount"
)

fig.update_traces(textposition="outside")
fig.update_layout(
    xaxis_title="Year",
    yaxis_title="Total Sales Amount",
    uniformtext_minsize=8,
    uniformtext_mode="hide"
)

fig.show()
```



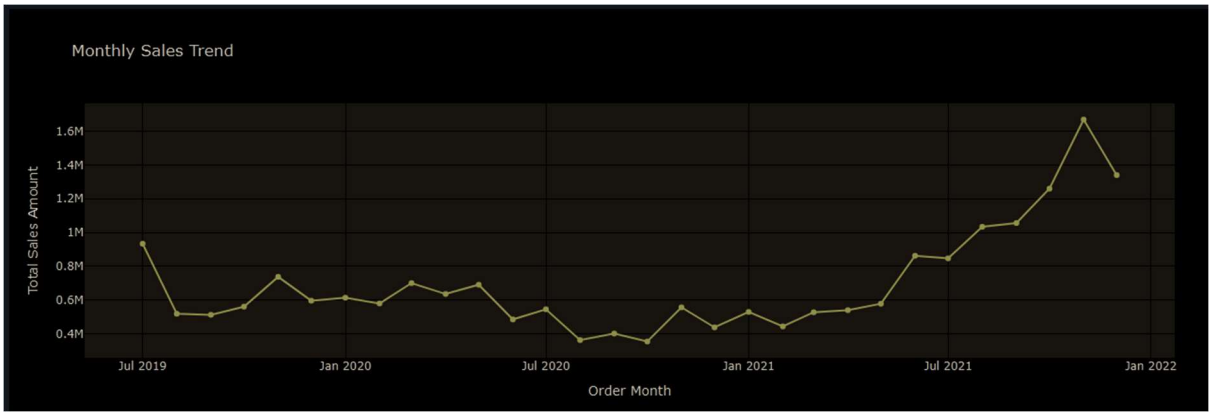
2. **Monthly Sales Trend (Line Chart)**

```
df_monthly = spark.table("adbrag.project4_schema.gold_monthly_sales")
pdf_monthly = df_monthly.orderBy("OrderMonth").toPandas()

fig = px.line(
    pdf_monthly,
    x="OrderMonth",
    y="TotalSalesAmount",
    markers=True,
    title="Monthly Sales Trend"
)

fig.update_layout(
    xaxis_title="Order Month",
    yaxis_title="Total Sales Amount"
)

fig.show()
```



3. **Top 10 Products by Sales**

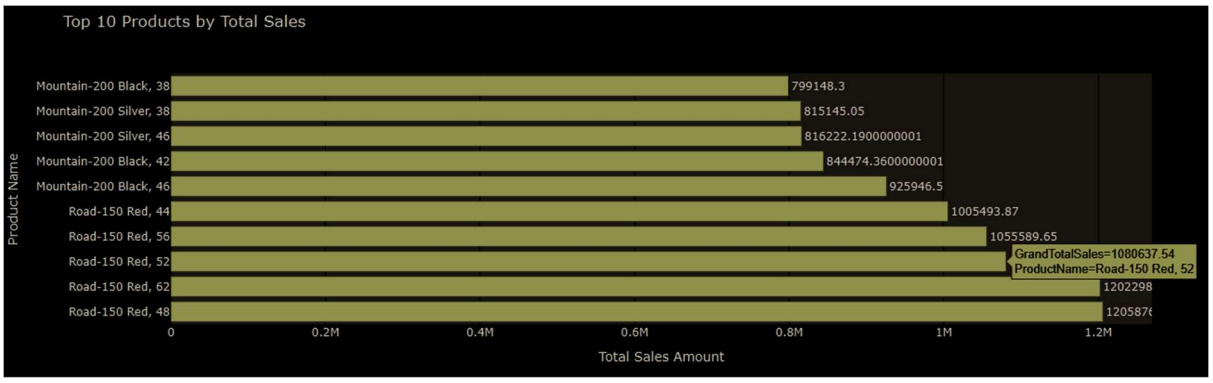
```
df_top_products = (
    spark.table("adbrag.project4_schema.gold_product_year_sales")
    .groupBy("ProductName")
    .agg(sum("TotalSalesAmount").alias("GrandTotalSales"))
    .orderBy(col("GrandTotalSales").desc())
    .limit(10)
)

pdf_top_products = df_top_products.toPandas()
```

```
fig = px.bar(
    pdf_top_products,
    x="GrandTotalSales",
    y="ProductName",
    orientation="h",
    text="GrandTotalSales",
    title="Top 10 Products by Total Sales"
)

fig.update_traces(textposition="outside")
fig.update_layout(
    xaxis_title="Total Sales Amount",
    yaxis_title="Product Name"
)

fig.show()
```



7. ADF Pipeline (Orchestration)

Pipeline Name

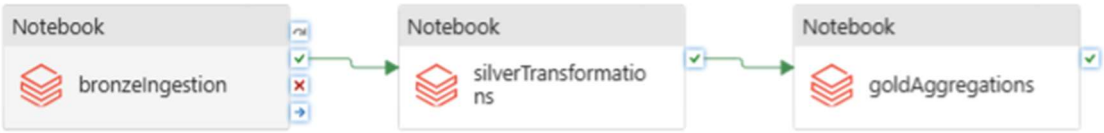
P4_masterPipeline

Activities

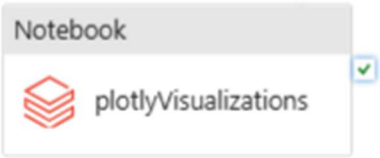
1. bronzeIngestion: Ingestion Notebook
2. silverTransformation: Bronze → Silver Notebook
3. goldAggregations: Silver → Gold Notebook
4. plotlyVisualizations → Plotly Notebook

ETL Flow

Ingestion → Silver → Gold



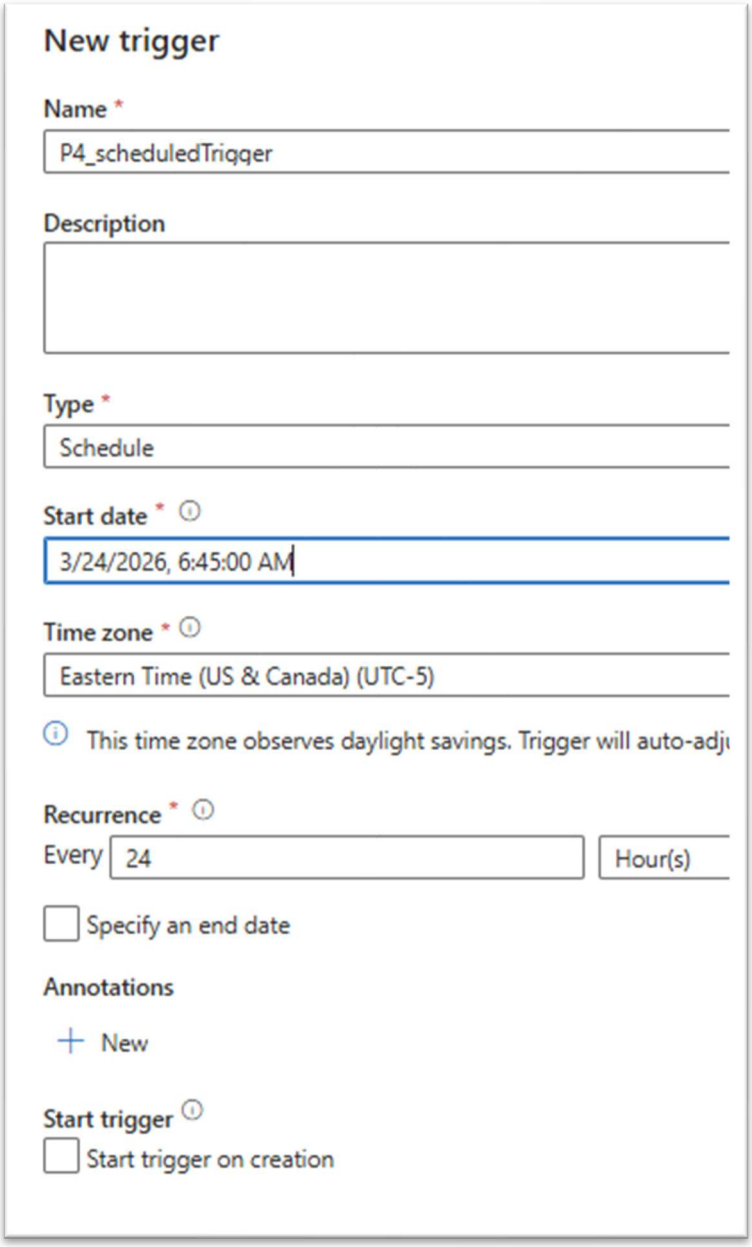
Visualization Notebook



Schedule Trigger

- Schedule Trigger (Daily execution)

To execute master pipeline every 24 hours.



8. Data Model

Bronze

- Raw CSV files in Volume

Silver

- Cleaned Delta table
- Standardized schema

Gold

- Aggregated business tables

9. Challenges & Solutions

Challenge	Solution
Cannot read GitHub URL directly	Used Python requests to download
Missing column headers	Stored schema in metadata table
Volume path issues	Correct schema + volume context
Product name containing commas	Used proper CSV quoting

10. Conclusion

This project demonstrates a complete end-to-end data engineering pipeline using Azure services. The implementation follows best practices such as Medallion architecture, metadata-driven design, and scalable orchestration using ADF.

- Metadata-driven pipeline design
- Dynamic schema assignment
- Unity Catalog Volume usage
- Multiple Gold aggregations
- End-to-end orchestration using ADF
- Visualization using Plotly